SVEUČILIŠTE U ZAGREBU
FAKULTET ORGANIZACIJE I INFORMATIKE
VARAŽDIN

Zdravko Blažević

ALGORITAM ZA PRONALAŽENJE NAJUDALJENIJIH ČVORAVA U STABLU

SEMINARSKI RAD

Varaždin, 2025.

SVEUČILIŠTE U ZAGREBU

FAKULTET ORGANIZACIJE I INFORMATIKE

V A R A Ž D I N

Zdravko Blažević

ALGORITAM ZA PRONALAŽENJE NAJUDALJENIJIH ČVORAVA U STABLU

SEMINARSKI RAD

Mentor :

Prof. dr. sc. Alen Lovrenčić

Varaždin, veljača 2025.

Sadržaj

1. Uvod 1

2. Teorija grafova 2

2.1. Tipovi grafova 3

2.1.1. Matrica susjedstva 4

2.1.2. Popis susjeda 4

3. Stabla 6

3.1. Binarna stabla 6

3.1.1. Pretraga i ophodnje stabala 7

3.2. Korištenje stabala 7

4. Algoritam za pretragu najudaljenijeg čvora u stablu 9

4.1. Implementaija najuadljenijeg čvora u stablu 10

5. Izračun vremenske složenosti 14

5.1. Složenost BFS 14

5.2. Složenost FindLongestPath 16

5.3. Složenost AddNode 16

5.4. Složenost AddEdge 17

5.5. Složenost main 18

5.6. Složenost Tree 19

5.7. Ukupna vremenska složenost 19

6. Zaključak 21

Popis literature 22

Popis slika 23

Popis popis tablica 24

1. Uvod

Stabla su jedna od najbitnijih struktura podataka u računalnoj znanosti jer omogucuju organizaciju i obradu podataka zbog svoje hijerarhijske strukture. Primjenjuje se u različitim područjima od pretrage i indeksiranja podataka do oblikovanja hijerarhijski suvisliih veza.

Jedan od problema vezan uz stabla je pronalazak najudaljenijeg para čvorova, odnosno para čvorova između kojh se nalazi najveć broju bridova u stablu. Na taj se način omogucava određivanje promrera stabla koji se definira kao njegova maksimalna širina. Algoritam za pronalazak najudaljenijeg para čvorova temelji se na pretraživanju stabla u širin i u dubinu (BFS ili DFS), a u samom algoritmu korišteno je pretraživanje u širinu.

Algoritam funkcionira u dva koraka. Prvo, pretraživanjem stabla pronalazi se najudaljeniji čvor od početnog čvora. Nakon toga, algoritam pokreće novo pretraživanje iz tog čvora kako bi pronašao drugi najudaljeniji čvor. Na kraju, algoritam pronalazi udaljenost između tih čvorova, te ispisuje koji su to čvorovi te izračunava promjer stabla.

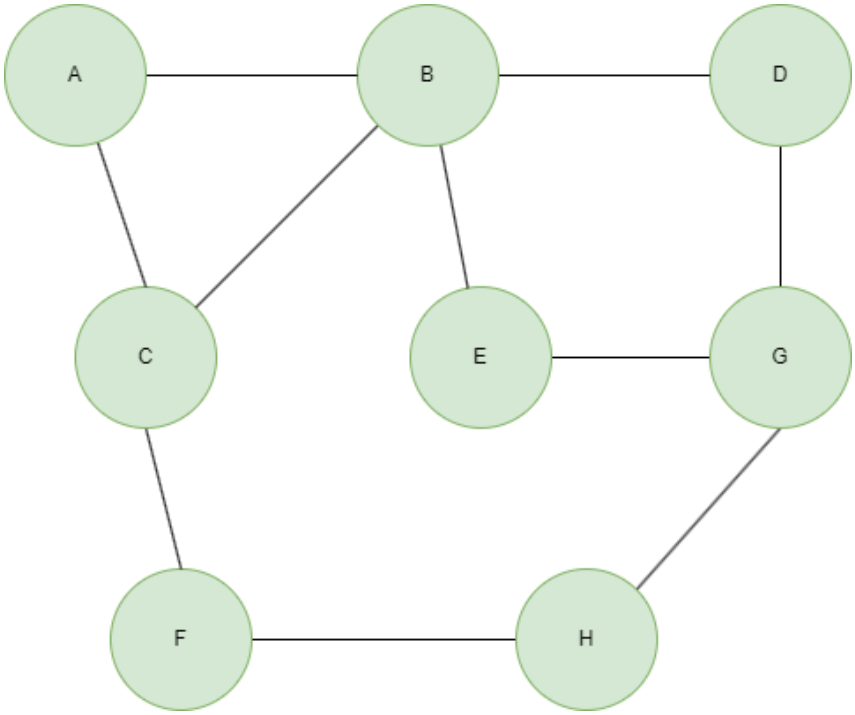
U samom seminaru će biti obrađena sama teorija stabala, ali će se prvo obraditi teorija grafa kako bi se mogla razumijeti šira slika koja stoji iza samg stabla, te kako bi konteskt samog algortma bio razumnjviji. Posbna pažnja samog rada će biti na stabu kao tipu grafoa, njihovoj strukturi, svojstvima i osnovnim operacijama. Nakon uvoda u teoriju, fokus rada bit će na implementaciji algoritma koji koristi pretragu u širinu (BFS) za pronalazak najudaljenijih čvorova u stablu i određivanje njegovog promjera.

Na kraju rada biti će izračunata složenosti samog algortima. Uz teoriju, rad će uključivati i primjere koji će dodatno ilustrirati način rada samog algoritma.

2. Teorija grafova

Iako će se u radu analizirati stablo, bitno je razumjeti nešto i o samom grafu, budući da je stablo poseban tip grafa. Stabla su usko povezana s grafovima, zato je korisno ukratko dotaknuti se same teorije grafova kako bi se bolje razumjeli strukturu i svojstva stabala.

Grafovi su matematičke strukture koje se koriste za prikazivanje odnosa između objekata. Sastoje se od čvorova, koji predstavljaju same objekte, i bridova, koji povezuju te čvorove i prikazuju njihove odnose. Kako bi se lakše razumijela terija grafa, možemo je se prikazati na primjeru čvorova koji mogu biti gradovi, ljudi, stanice u mreži ili bilo koji drugi objekti, dok bridovi mogu predstavljati ceste, kablove ili druge vrste poveznica. Grafove se također mogu zamisliti kao mrežu točkica povezanih linijama. Na primjer, gradovi Zagreb, Split i Rijeka mogu se prikazati kao čvorovi, dok su ceste koje ih povezuju bridovi. Ako postoji cesta između Zagreba i Splita, zanči da postoji brid između čvorova koji predstavljaju ta dva grada. [1]



Slika 1: Primjer grafa

Čvorovi (računala): V=$\{A, B, C, D, E, F, G, H\}$

Bridovi (poveznice): E=$\{(A, B), (A, C), (B, D), (B, E), (C, F), (D, G), (E, G), (F, H), (G, H)\}$

Također, postoje još i Petlje koje posebna vrsta brida u grafu gdje brid započinje i završava na istom čvoru, što znači da je čvor povezan sam sa sobom. Na primjer, da na grafu sa slike 1. postoji brid (A,A) on bi predstavljao petlju na čvoru A, što vizualno izgleda kao kružnica nacrtana oko tog čvora.

Staza je niz bridova u grafu gdje su bridovi međusobno povezani, ali čvorovi na stazi mogu biti posjećeni više puta.

Promjer stabla je najduži put između bilo koja dva čvora u stablu mjeri se brojem bri-

dova. Promjer stabla prikazuje najveću udaljenost unutar stabla.

Put je posebna vrsta staze u kojoj se svaki čvor ponavlja samo jednom, bez dodatnog ponavljanja. Za razliku od staze, gdje se čvorovi i bridovi mogu ponoviti, put je najkraći ili jedinstveni prolaz kroz graf između dva čvor.

Primjer sa slike 1:

- **Staza** između čvorova A i H:

$$A \to B \to E \to G \to B \to A \to C \to F \to H$$

  U ovoj stazi čvorovi B i A se ponavljaju, pa nije put.

- **Put** između čvorova A i H:

$$A \to B \to E \to G \to H$$

  U ovom slučaju, svaki čvor se posjećuje samo jednom.

2.1. Tipovi grafova

Grafovi dolaze u različitim vrstama, ovisno o karakteristikama njihovih bridova i čvorova, kao i o tome kako su povezani. Ovdje su osnovni tipovi grafova:

Jednostavan graf je graf u kojem nema višestrukih bridova između istih čvorova niti petlji, svaki brid je povezan sa različitim čvorovima, a između dva čvora može postojati samo jedan brid. Jednostavni grafovi se koriste za modeliranje odnosa gdje su veze jedinstavnije.[1]

Generalni graf je tip grafa koji dopušta višestruke veze među istim čvorovima, kao i petlje. Višestruki bridovi prikazuju različitih tipova veza između identičnih čvorova, dok petlje prikazuju situacije u kojima se čvor sam sa sobom povezuje.[1]

Povezani graf je graf u kojem postoji barem jedan put između svakog para čvorova. To znači da nema izoliranih čvorova, a sve komponente grafa čine jednu cjelinu. [1]

Potpuni graf je graf u kojem je svaki čvor povezan bridom sa svakim drugim čvorom. Ako graf ima n čvorova, broj bridova u potpunom grafu je $\frac{n(n-1)}{2}$. Takav graf se koristi za prikaz maksimalne povezanosti, gdje su svi elementi međusobno izravno povezani, poput mreža u kojima je svako računalo izravno povezano sa svakim drugim računalom.[1]

Stablo je poseban tip povezanog grafa koji nema ciklusa. Između svakog para čvorova u stablu postoji točno jedan put, a broj bridova u stablu s n čvorova je uvijek $n - 1$.[1]

Šuma je skup nepovezanih stabala. To znači da se sastoji od više komponenti, gdje je svaka komponenta stablo. [1]

2.1.1. Matrica susjedstva

Matrica susjedstva jedan je od osnovnih načina za prikaz grafa pomoću kvadratne matrice. U njoj redci i stupci odgovaraju čvorovima grafa, dok vrijednosti unutar matrice predstavljaju povezanost između tih čvorova. Ako su dva čvora povezana bridom, odgovarajući element matrice ima vrijednost 1 dok 0 označava da između čvorova ne postoji brid. Dimenzije matrice susjedstva ovise o broju čvorova u grafu; za graf s n čvorova, matrica je dimenzija n×n [1]. UTablica 1. predstavlja matricu susjedstva za graf koji se nalazi na slici 1.

Tablica 1: Matrica susjedstva grafa iz grafa sa slike 1.

	A	B	C	D	E	F	G	H
A	0	1	1	0	0	0	0	0
B	1	0	0	1	1	0	0	0
C	1	0	0	0	0	1	0	0
D	0	1	0	0	0	0	1	0
E	0	1	0	0	0	0	1	0
F	0	0	1	0	0	0	0	1
G	0	0	0	1	1	0	0	1
H	0	0	0	0	0	1	1	0

Matrica susjedstva prikazuje odnose među čvorovima neusmjerenog grafa s osam čvorova označenih kao A, B, C, D, E, F, G, H. Elementi matrice s vrijednošću 1 označavaju da postoji brid između dva čvora, dok vrijednost 0 pokazuje da takva veza ne postoji. Simetričnost matrice prikazuej da se radi o neusmjerenom grafu, jer brid između A i B automatski znači i brid između B i A.

Dijagonalni elementi matrice imaju vrijednost 0, što znači da graf nema petlji, nema bridova koji spajaju čvorove same sa sobom. Graf je povezan, jer između svakog para čvorova postoji barem jedan put, bilo direktno ili preko drugih čvorova, ali graf nije potpun, jer nisu svi čvorovi međusobno izravno povezani.

2.1.2. Popis susjeda

Tablica 2: Popis susjedstva grafa

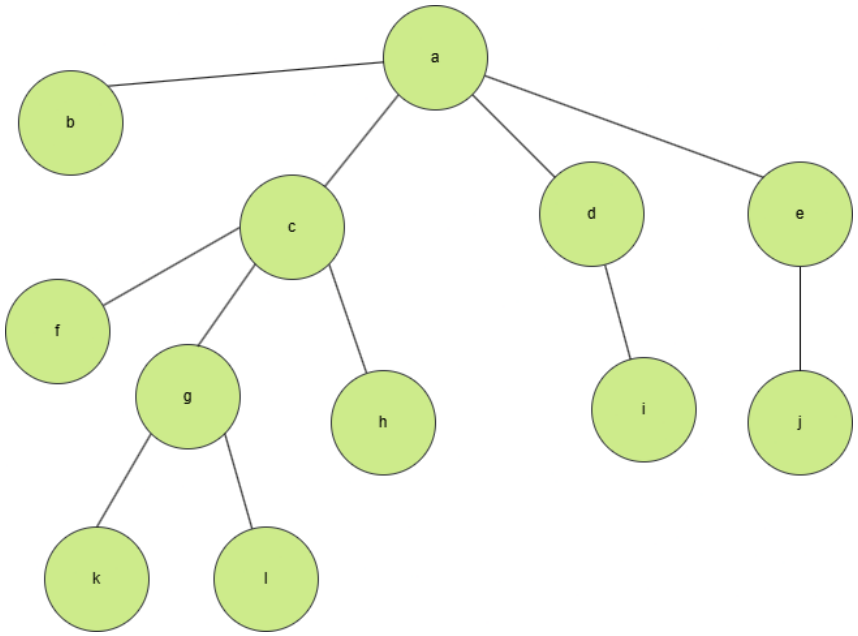
Čvor	**Susjedi**
A	B, C
B	A, D, E
C	A, F
D	B, G
E	B, G
F	C, H
G	D, E, H
H	F, G

Popis susjedstva je način prikaza grafa u kojem se za svaki čvor navodi popis svih čvorova s kojima je povezan bridovima. Svaki čvor u grafu ima svoju listu susjeda, koja predstavlja

sve čvorove s kojima je izravno povezan.[1]

3. Stabla

Stabla su osnovne dinamičke strukture podataka koje se koriste za prikaz hijerarhijskih odnosa među objektima. Sastoje se od čvorova povezanih granama, pri čemu svaki čvor, osim korijenskog, ima točno jednog roditelja. Korijenski čvor nema roditelja i predstavlja polaznu točku stabla, dok su listovi čvorovi bez potomaka. Ostali čvorovi nazivaju se unutrašnjim čvorovima i mogu imati jedno ili više djece. Zbog svojih svojstava, stabla su idealna za modeliranje hijerarhija, poput obiteljskih stabala, logičkih izraza ili struktura podataka u računalstvu. [2] Na slici 2. vidljiv je primjer stabla:



Slika 2: Primjer stabla

Visina stabla predstavlja najveću udaljenost od korijena stabla do bilo kojeg lista, odnosno najudaljenijeg čvora od korijena. Na primjeru sa slike 2. visina stabla je 4 jer najduži put od korijena čvora a do nekog lista vodi do čvora k ili l i sadrži 4 bridova ($a \to c \to g \to k$).

S druge strane, **promjer stabla** označava najveću udaljenost između bilo koja dva čvora u stablu, mjereno brojem bridova. Na slici 2, promjer stabla je 6 jer najduži put u stablu vodi od čvora k do čvora j i prelazi preko čvorova $k \to g \to c \to a \to e \to j$.

3.1. Binarna stabla

U uređenom binarnom stablu podaci su organizirani prema određenoj relaciji poretka, koja može biti, primjerice, uobičajeno rastuće ili opadajuće uređenje za brojeve. Za svaki čvor n u uređenom binarnom stablu vrijedi da su vrijednosti podataka u čvorovima njegovog lijevog podstabla manje ili jednake vrijednosti u čvoru n, dok su vrijednosti podataka u čvorovima njegovog desnog podstabla veće od vrijednosti u čvoru n. [3]

Organizacija podataka u uređenom binarnom stablu omogućava jednostavniju obradu

podataka zbog korištenja binarne pretrage, ali se ta učinkovitost može izgubiti ako se binarno stablo degenerira, primjerice kada svaki čvor ima samo jednog potomka, ili ako postane gotovo degenerirano. U takvim slučajevima, složenost operacija dodavanja i pretrage elemenata postaje linearna u odnosu na broj elemenata u stablu. [4]

Binarno stablo je podatkovna struktura u kojoj svaki čvor može imati najviše dva potomka, nazvana lijevo i desno dijete, te se tom strukturom omogućava bolja i kvalitetna organizacija podataka i podrška za operacije poput umetanja, pretrage i brisanja. Svaki čvor stabla ima oznaku i može biti povezan sa svojim roditeljskim čvorom ili potomcima.

Prema teoriji grafova, binarna stabla su posebna vrsta korijenskih stabala, a njihova se svojstva koriste za brojne algoritme i implementacije. Na primjer, razina čvora definirana je kao duljina puta od korijena do tog čvora, dok se visina stabla mjeri kao najveća razina čvorova u stablu. Potpuno binarno stablo, u kojem su svi čvorovi popunjeni na svakoj razini, ima $2^h + 1 - 1$ čvorova, gdje je h visina stabla.[3] [4]

3.1.1. Pretraga i ophodnje stabala

Postoji nekoliko načina prolaska kroz binarno stablo:

- **Preorder (prvo korijen)** – obrada čvora prije njegove djece.
- **Inorder (u sredini)** – obrada lijevog podstabla, zatim čvora, a potom desnog podstabla.
- **Postorder (na kraju)** – obrada djece prije obrade čvora.

Te se ophodnje najčešće izvode rekurzivno, ali se mogu implementirati i iterativno uz korištenje pomoćnih struktura poput stoga. Također ove metode se koriste za različite potrebe obrade podataka i imaju vremensku složenost $O(n)$. [3]

3.2. Korištenje stabala

Stabla igraju važnu ulogu za strukturu samih podataka u računalnim znanostima zbog svojih mogućnosti da organiziraju i upravljaju informacijama. U kompilatorima se stabla, poput sintaksnih stabala, koriste za parsiranje i analizu programskog koda, pretvarajući linearni niz tokena u strukturu koja odražava hijerarhiju naredbi i izraza.

U bazama podataka, stabla poput B-stabala i B+ stabala koriste se za organizaciju i pretraživanje podataka, te se omogućava dohvaćanje i manipulaciju velikih količina podataka, što ih čini idealnim za uporabu u relacijskim bazama podataka i indekse datotekama.

Stabla također imaju veliku ulogu u datotečnim sustavima. Hijerarhijska struktura direktorija koristi se za organizaciju datoteka i mapa, omogućujući lakše pronalaženje i upravljanje.

Jedan od najprepoznatljivijih primjera korištenja stabala u svakodnevnoj tehnologiji je struktura direktorija na računalima, te takva hijerarhijska organizacija datoteka omogućuje korisnicima logičnu organizaciju podataka u mape i podmape. Na primjer, mapa "Dokumenti"

može sadržavati podmape poput "Projekti" i "Računi", unutar kojih mogu postojati dodatne razine organizacije."

4. Algoritam za pretragu najudaljenijeg čvora u stablu

Najudaljeniji čvorovi u stablu su oni između kojih postoji najveći broj bridova, a ta udaljenost naziva se promjer stabla. Pošto stablo nema ciklusa, između svakog para čvorova postoji jedan put, što znači da je moguće jednostavno izračunati udaljenost. Promjer stabla može se pronaći pomoću pretraživanja u dubinu (DFS) ili pretraživanja u širinu (BFS), a postupak obično započinje iz proizvoljnog čvora, te traži najudaljeniji čvor od početnog, a zatim se pretraga ponavlja iz tog čvora kako bi se pronašao drugi najudaljeniji čvor i time odredio promjer stabla.

Pretraživanje u dubinu (DFS) istražuje stablo što dublje u jednom smjeru prije nego što se vrati unazad i istraži preostale grane. DFS koristi rekurziju ili stog za praćenje posjećenih čvorova. Algoritam DFS može se koristiti za pronalaženje promjera stabla na isti način kao BFS, ali zahtijeva više memorije zbog rekurzije. [5]

Pretraživanje u širinu (BFS), koje je korišteno u ovo sminarskom radu je algoritam za pretraživanje grafa ili stabla koji istražuje čvorove sljedno, počevši od početnog čvora, zatim prelazi na njegove obližnje susjede, potom na susjede tih čvorova, i tako dalje, dok se svi čvorovi ne posjete. BFS koristi red za praćenje čvorova koje treba obraditi, te se na taj način svi čvorovi na istoj dubini istraže prije nego što algoritam prijeđe na dublje razine stabla.[5]

Algoritam za određivanje promjera stabla koristi dvije BFS pretrage i temelji se na činjenici da se jedan kraj promjera može pronaći pokretanjem BFS-a iz proizvoljnog čvora. Funkcija `FindLongestPath` implementira ovaj algoritam u dva koraka.

Prvo se pokreće BFS iz čvor `1` kako bi se pronašao najudaljeniji čvor u stablu, te se na taj način osigrava da će pronađeni čvor biti jedan od krajeva promjera stabla. Zatim se pokreće druga BFS pretraga iz tog pronađenog čvora kako bi se pronašao drugi najudaljeniji čvor i time odredila njihova udaljenost, koja predstavlja promjer stabla.

Funkcija `BFS` koristi pretragu u širinu. Inicijalizira polja `posjeceni`, `udaljenosti` i red `red`, pri čemu se svi čvorovi postavljaju kao neposjećeni, a početni čvor dodaje se u red. Glavna `while` petlja obrađuje čvorove jedan po jedan pretražujući njhove oližnje susjede i dodajući ih u red ako još nisu posjećeni. Kada algoritam završi, u varijabli `najdaljiCvor` nalazi se čvor koji je najudaljeniji od početnog, a njegova udaljenost spremljena je u `maksUdaljenost`.

Kada se prva BFS pretraga završi, funkcija `FindLongestPath` koristi pronađeni najudaljeniji čvor kao novi početni čvor za drugu BFS pretragu, koja ponovno prolazi kroz cijelo stablo i određuje čvor najudaljeniji od njega. Udaljenost između ta dva čvora predstavlja promjer stabla.

U Tablici 3. prikazan je popis funkcija koje će biti korištene u kodu za pronalaženje najduljeg puta u stablu. Tablica sadrži nazive funkcija i njihove opise kako bi se jasno razumjela njihova svrha i način rada unutar implementacije.

Tablica 3: Prikaz funkcija algoritma

Naziv funkcija	Opis
`Tree()`	Inicijalizira stablo, postavlja sve čvorove kao nezauzete i sve veze u matrici susjedstva na `false`.
`AddNode(indeks, oznaka)`	Dodaje čvor s danim indeksom i oznakom u stablo. Ako čvor već postoji, ispisuje grešku i prekida program.
`AddEdge(u, v)`	Dodaje brid između čvorova u i v. Ako bilo koji od čvorova ne postoji, ispisuje grešku i prekida program.
`BFS(pocetak, najdaljiCvor, maksUdaljenost)`	Provodi pretragu u širinu (BFS) počevši od čvora $pocetak$. Ažurira najudaljeniji čvor i njegovu udaljenost.
`FindLongestPath()`	Pronalazi promjer stabla pomoću dvostruke BFS pretrage i ispisuje rezultate.

4.1. Implementaija najuadljenijeg čvora u stablu

U nastavku se nalazi implementacija koda koji pronalazi najudaljeniji put u stablu, poznat kao promjer stabla. Kod koristi pretragu u širinu (BFS) kako bi odredio dva najudaljenija čvora u stablu i njihovu udaljenost.

```
#include <iostream>

#ifndef MAXN
#define MAXN 100
#endif

class Tree {
private:
    struct Cvor {
        int oznaka;
        bool zauzet;
    };

    Cvor cvorovi[MAXN + 1];
    bool veze[MAXN + 1][MAXN + 1];

public:
    typedef int NodeIndex;
    const NodeIndex nepostojeci = 0;
    Tree() {
        for (int i = 1; i <= MAXN; i++) {
            cvorovi[i].zauzet = false;
            for (int j = 1; j <= MAXN; j++) {
                veze[i][j] = false;
            }
        }
    }
```

```
void AddNode(NodeIndex indeks, int oznaka) {
    if (cvorovi[indeks].zauzet) {
        std::cout << "Čvor␣ćve␣postoji!" << std::endl;
        exit(EXIT_FAILURE);
    }
    cvorovi[indeks].zauzet = true;
    cvorovi[indeks].oznaka = oznaka;
}

void AddEdge(NodeIndex u, NodeIndex v) {
    if (!cvorovi[u].zauzet || !cvorovi[v].zauzet) {
        std::cout << "Jedan␣od␣čvorova␣ne␣postoji!" << std::endl;
        exit(EXIT_FAILURE);
    }
    veze[u][v] = true;
    veze[v][u] = true;
}

void BFS(NodeIndex pocetak, NodeIndex &najdaljiCvor, int &maksUdaljenost) {
    bool posjeceni[MAXN + 1] = {false};
    int udaljenosti[MAXN + 1] = {0};
    int red[MAXN + 1];
    int prvi = 0, zadnji = 0;

    red[zadnji++] = pocetak;
    posjeceni[pocetak] = true;

    maksUdaljenost = 0;
    najdaljiCvor = pocetak;

    while (prvi < zadnji) {
        NodeIndex trenutni = red[prvi++];
        for (int i = 1; i <= MAXN; i++) {
            if (veze[trenutni][i] && !posjeceni[i]) {
                posjeceni[i] = true;
                udaljenosti[i] = udaljenosti[trenutni] + 1;
                red[zadnji++] = i;

                if (udaljenosti[i] > maksUdaljenost) {
                    maksUdaljenost = udaljenosti[i];
                    najdaljiCvor = i;
                }
            }
        }
    }
}

void FindLongestPath() {
    NodeIndex prviCvor, drugiCvor;
    int udaljenostPrvi, udaljenostDrugi;

    BFS(1, prviCvor, udaljenostPrvi);
```

```
        BFS(prviCvor, drugiCvor, udaljenostDrugi);

        std::cout << "Najdalji␣čvorovi␣su:␣" << prviCvor << "␣i␣" << drugiCvor <<
            std::endl;
        std::cout << "Promjer␣stabla␣je:␣" << udaljenostDrugi << std::endl;
    }
};

int main() {
    Tree stablo;
    int brojCvorova;

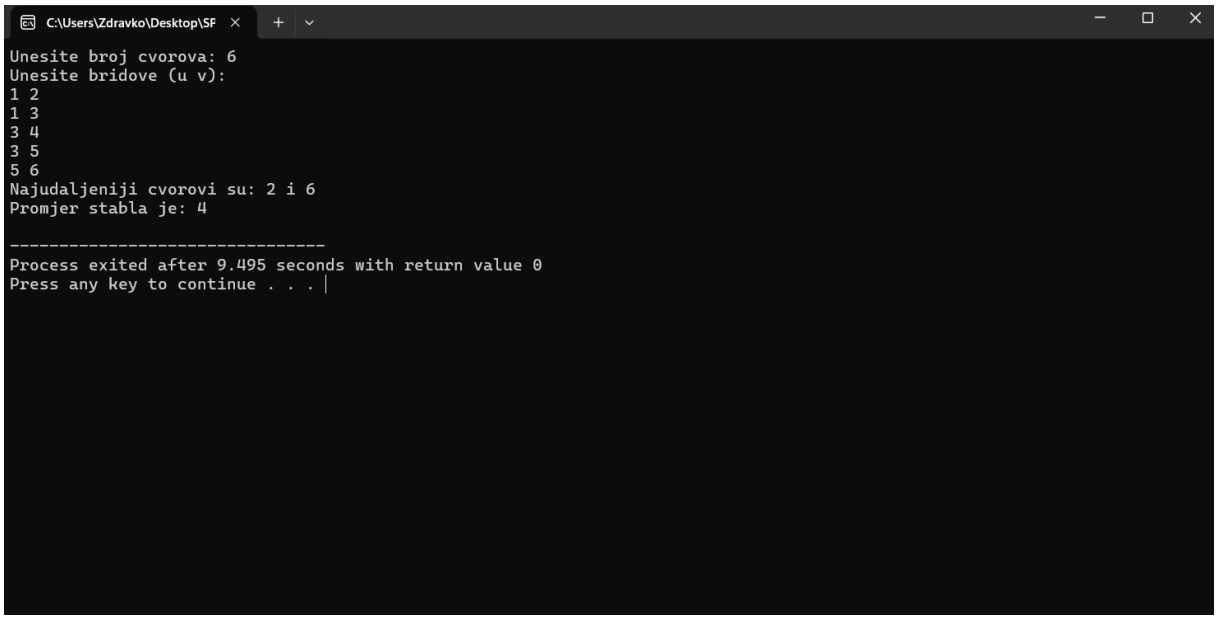
    std::cout << "Unesite␣broj␣čvorova:␣";
    std::cin >> brojCvorova;

    for (int i = 1; i <= brojCvorova; i++) {
        stablo.AddNode(i, i);
    }

    std::cout << "Unesite␣bridove␣(u␣v):" << std::endl;
    for (int i = 0; i < brojCvorova - 1; i++) {
        int u, v;
        std::cin >> u >> v;
        stablo.AddEdge(u, v);
    }

    stablo.FindLongestPath();

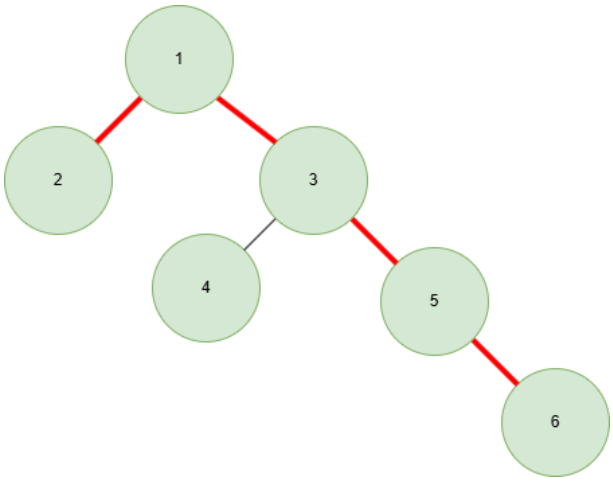
    return 0;
}
```



Slika 3: Rezultat pokretanja programa

Slika 3. prikazuje pokrenuti program, dok slika 4. prikazuje graf iz primjera na kojoj je

put između dva naudaljenija čvora u satblu prikazan crvenom bojom.



Slika 4: Prikaz grafa iz primjera

5. Izračun vremenske složenosti

Složenost algoritma odnosi se na procjenu resursa potrebnih za izvođenje programa. Postoje dvije osnovne vrste složenosti: vremenska složenost, koja mjeri broj operacija potrebnih za izvršavanje algoritma, i prostorna složenost, koja se odnosi na količinu memorije koju algoritam zauzima. Za potreba ovog seminara biti će izračunata vremenska slženost.

Razlikuju se više vrsta vremeskih složenosti, među kojima su: [6]

- **Konstantna složenost ($O(1)$)**: Algoritam zahtijeva fiksni broj operacija, neovisno o veličini ulaznih podataka.
- **Logaritamska složenost ($O(\log n)$)**: Problem se postupno reducira smanjivanjem broja elemenata za fiksni omjer, npr. binarna pretraga.
- **Linearna složenost ($O(n)$)**: Svaki ulazni element zahtijeva određenu operaciju, što je čest slučaj kod jednostavnih algoritama pretraživanja ili obrade podataka.
- **Kvadratna složenost ($O(n^2)$)**: Algoritmi koji koriste dvostruke petlje ili prolaze kroz svaki par elemenata, poput jednostavnih algoritama sortiranja, imaju kvadratni rast.
- **Eksponencijalna složenost ($O(2^n)$)**: Pojavljuje se kod algoritama koji rješavaju probleme s puno kombinacija, poput Hanojskih tornjeva, i često je nepraktična za velike vrijednosti n.

U nastavku će biti detaljnije analizirane pojedinačne funkcije i dijelovi koda kako bi se izračunala vremenska složenost pojedinih dijelova koda.

5.1. Složenost BFS

Funkcija `BFS` implementira pretragu u širinu (*Breadth-First Search*) kako bi pronašla najudaljeniji čvor i njegovu udaljenost od početnog čvora. Budući da se pretraga izvodi nad stablu, broj čvorova označavamo s n, dok je broj bridova uvijek $n - 1$.

Na početku, funkcija inicijalizira tri polja veličine $O(n)$, jedno za označavanje posjećenih čvorova, drugo za pohranu udaljenosti od početnog čvora i treće za redoslijed obrade cvorova tijekom BFS pretrage. Inicijalizacija polja vrši se nad svim čvorovima, ukupna složenost ovog dijela je $O(n)$.

Nakon inicijalizacije, početni čvor se stavlja u red i označava kao posjećen, što su operacije $O(1)$. Zatim se pokreće glavna `while` petlja, koja se izvršava sve dok BFS red nije prazan. Budući da se svaki čvor dodaje u red samo jednom, ova petlja prolazi kroz svih n čvorova, što znači da ukupno ima $O(n)$ iteracija.

Unutar glavne petlje nalazi se `for` petlja koja iterira kroz susjede trenutnog čvora. U stablu svaki čvor može imati najviše $O(n)$ susjeda u ekstremnom slučaju, ali ukupan broj prolazaka kroz sve bridove ne može biti veći od $O(n - 1)$. Stoga se unutarnja petlja ukupno izvrši $O(n)$ puta.

Unutar `if` uvjeta provjerava se postoji li veza i je li čvor već posjećen, što traje $O(1)$. Ako uvjet prođe, ažuriraju se polja `posjeceni`, `udaljenosti` i čvor se dodaje u red, što su također operacije $O(1)$. Na kraju se provjerava i ažurira najudaljeniji čvor, što traje $O(1)$. Budući da se ove operacije izvršavaju za svakog susjeda, njihov ukupan broj odgovara broju iteracija unutarnje petlje, što znači da ih može biti najviše $O(n)$.

U nastavku će biti izračunata vremenska složenost algortima:

```
void BFS(NodeIndex pocetak, NodeIndex &najdaljiCvor, int &maksUdaljenost) {
    bool posjeceni[MAXN + 1] = {false};               // a1
    int udaljenosti[MAXN + 1] = {0};                  // a2
    int red[MAXN + 1];                                // a3
    int prvi = 0, zadnji = 0;                         // a4

    red[zadnji++] = pocetak;                          // a5
    posjeceni[pocetak] = true;                        // a6

    maksUdaljenost = 0;                               // a7
    najdaljiCvor = pocetak;                           // a8

    while (prvi < zadnji) {                           //<= *(n), a9
        NodeIndex trenutni = red[prvi++];             // b1
        for (int i = 1; i <= MAXN; i++) {             // <= *(n),b2
            if (veze[trenutni][i] && !posjeceni[i]) {// c1
                posjeceni[i] = true;                  // e1
                udaljenosti[i] = udaljenosti[trenutni] + 1; // e2
                red[zadnji++] = i;                    // e3

                if (udaljenosti[i] > maksUdaljenost) { // e4
                    maksUdaljenost = udaljenosti[i]; //f1
                    najdaljiCvor = i; //f2
                }
            }
        }
    }
}
```

$$T_{\text{BFS}} = \underbrace{a_1 \cdot n}_{\text{Inicijalizacija polja}} + \underbrace{a_2}_{\text{Dodavanje početnog čvora u red}} + \underbrace{b_1 \cdot n}_{\text{Vanjska while petlja}} + \underbrace{b_2 \cdot (n-1)}_{\text{Unutarnja for petlja}} + \underbrace{c_1 \cdot (n-1)}_{\text{Provjera veze}}$$
$$+ \underbrace{(e_1 + e_2 + e_3) \cdot (n-1)}_{\text{Ažuriranje podataka}} + \underbrace{(e_4 + f_1 + f_2) \cdot (n-1)}_{\text{Provjera najudaljenijeg čvora}}$$

$$T_{\text{BFS}} = (a_1 + b_1) \cdot n + (b_2 + c_1 + e_1 + e_2 + e_3 + e_4 + f_1 + f_2) \cdot (n-1) + a_2$$

$$T_{\text{BFS}} = d_1 \cdot n + d_2 \cdot (n-1) + O(1)$$

Iz ovoga se može zaključiit da je složenost:

$$T_{\text{BFS}} = O(n)$$

5.2. Složenost FindLongestPath

Funkcija prvo deklarira nekoliko varijabli za pohranu rezultata pretrage, što je operacija složenosti $O(1)$. Nakon toga, pokreće se prva BFS pretraga iz korijena stabla, koja prolazi kroz svih n čvorova, pa je njena složenost $O(n)$.

Zatim se iz najudaljenijeg pronađenog čvora pokreće još jedna BFS pretraga s istom složenošću $O(n)$. Nakon izvršenja pretraga, funkcija ispisuje rezultate na ekran, što traje $O(1)$, jer su operacije ispisa konstantne složenosti. U nastavki je prikazan kod za funkciju FindLongestPath:

```
void FindLongestPath() {
    NodeIndex prviCvor, drugiCvor; // a1
    int udaljenostPrvi, udaljenostDrugi; // a2

    BFS(1, prviCvor, udaljenostPrvi); // <=*(n), a3
    BFS(prviCvor, drugiCvor, udaljenostDrugi); // <=*(n), a4

    std::cout << "Najdalji␣čvorovi␣su:␣" << prviCvor << "␣i␣" << drugiCvor << std::
        endl; // a5
    std::cout << "Promjer␣stabla␣je:␣" << udaljenostDrugi << std::endl; // a6
}
```

U nastavki je prikazan izračun složenosti operacije:

$$T_{\text{FindLongestPath}} = \underbrace{O(1)}_{\text{Deklaracija varijabli}} + \underbrace{O(n)}_{\text{Prvi BFS pretraga}} + \underbrace{O(n)}_{\text{Drugi BFS pretraga}} + \underbrace{O(1)}_{\text{Ispis rezultata}}$$

$$T_{\text{FindLongestPath}} = O(1) + O(n) + O(n) + O(1)$$

Iz ovoga se može zaključiit da je složenost:

$$T_{\text{FindLongestPath}} = O(n)$$

5.3. Složenost AddNode

Iako sljedeći kod radi sa strukturom podataka, a ova funkcija nije dio samog algoritma, ali njena složenost će ipak biti proučena u nastavku:

```
void AddNode(NodeIndex indeks, int oznaka) {
    if (cvorovi[indeks].zauzet) {                // a1
        std::cout << "Čvor␣će␣postoji!" << std::endl;  // b1
        exit(EXIT_FAILURE);                      // b2
    }
    cvorovi[indeks].zauzet = true;               // a2
    cvorovi[indeks].oznaka = oznaka;             // a3
}
```

Funkcija `AddNode` dodaje novi čvor u strukturu podataka. Prvo provjerava da li čvor već postoji u stablu, što traje $O(1)$ jer se radi o pristupu elementu niza. Ako čvor već postoji, ispisuje poruku i prekida program, a te operacije su također $O(1)$.

Ako čvor ne postoji, funkcija postavlja `cvorovi[indeks].zauzet = true` i dodjeljuje oznaku `cvorovi[indeks].oznaka = oznaka`, što su operacije konstantnog vremena $O(1)$.

$$T_{\text{AddEdge}} = \underbrace{O(1)}_{\text{Provjera postojanja čvorova}} + \underbrace{O(1)}_{\text{Ispis poruke i izlazak iz programa (ako je potrebno)}} + \underbrace{O(1)}_{\text{Dodavanje bridova u matricu susjedstva}}$$

$$T_{\text{AddEdge}} = O(1) + O(1) + O(1)$$

Iz ovoga se može zaključiit da je složenost:

$$T_{\text{AddEdge}} = O(1)$$

5.4. Složenost AddEdge

Iako sljedeći kod radi sa strukturom podataka, a ova funkcija nije izravni dio algoritma, njena složenost će ipak biti analizirana u nastavku:

```
void AddEdge(NodeIndex u, NodeIndex v) {
    if (!cvorovi[u].zauzet || !cvorovi[v].zauzet) {      // a1
        std::cout << "Jedan od čvorova ne postoji!" << std::endl;  // b1
        exit(EXIT_FAILURE);                                       // b2
    }
    veze[u][v] = true;                                    // a2
    veze[v][u] = true;                                    // a3
}
```

Funkcija `AddEdge` dodaje brid između dva čvora u matrici susjedstva. Prvo provjerava jesu li oba čvora zauzeta, što traje $O(1)$ jer se radi o jednostavnim pristupima elementima niza. Ako jedan od čvorova ne postoji, ispisuje poruku na ekran i prekida program, a te operacije su također $O(1)$.

Ako su oba čvora ispravna, funkcija postavlja `veze[u][v] = true` i `veze[v][u] = true`, što su također operacije $O(1)$ jer se radi o jednostavnim upisima u matricu.

$$T_{\text{AddEdge}} = \underbrace{O(1)}_{\text{Provjera čvorova}} + \underbrace{O(1)}_{\text{Ispis poruke}} + \underbrace{O(1)}_{\text{Izlazak iz programa}} + \underbrace{O(1)}_{\text{Dodavanje brida}}$$

$$T_{\text{AddEdge}} = O(1) + O(1) + O(1) + O(1)$$

$$T_{\text{AddEdge}} = O(1)$$

5.5. Složenost main

Unastavku je prikazana mian fukcija koja pokreće cijeli program, omogućava unos broja čvorova i bridova te poziva funkciju `FindLongestPath`, za koju će također biti izračunata složenost u nastavku:

```
int main() {
    Tree stablo; // a1
    int brojCvorova; // a2

    std::cout << "Unesite broj čvorova: "; // a3
    std::cin >> brojCvorova; // a4

    for (int i = 1; i <= brojCvorova; i++) { // <=*(n), a5
        stablo.AddNode(i, i); // b1
    }

    std::cout << "Unesite bridove (u v):" << std::endl; // a6
    for (int i = 0; i < brojCvorova - 1; i++) {  // (izvrava se n-1 puta), a7
        int u, v; // b2
        std::cin >> u >> v; // b3
        stablo.AddEdge(u, v); // b4
    }

    stablo.FindLongestPath(); //a8

    return 0; //a9
}
```

Funkcija `main` inicijalizira stablo i omogućava unos broja čvorova n. Prvo dodaje n čvorova, gdje svaka operacija traje $O(1)$, pa je ukupna složenost ovog dijela $O(n)$. Zatim korisnik unosi $n-1$ bridova, jer stablo s n čvorova ima tačno $n-1$ veza. Svaka iteracija unosa i dodavanja veze traje $O(1)$, pa ukupna složenost ostaje $O(n)$.

Nakon unosa, poziva se funkcija `FindLongestPath`, koja koristi dvije BFS pretrage. Budući da BFS prolazi kroz svih n čvorova, njegova složenost je $O(n)$, pa se ukupna složenost ovog dijela računa kao:

$$T_{\text{main}} = a_1 + a_2 + a_3 + a_4 + a_5 \cdot n + b_1 \cdot n + a_6 + a_7 \cdot (n-1) + (b_2 + b_3 + b_4) \cdot (n-1) + a_8 + O(n) + a_9$$

$$T_{\text{main}} = (a_1 + a_2 + a_3 + a_4 + a_6 + a_8 + a_9) + (a_5 + b_1) \cdot n + (a_7 + b_2 + b_3 + b_4) \cdot (n-1) + O(n)$$

$$T_{\text{main}} = \underbrace{O(1)}_{\text{Inicijalizacija}} + \underbrace{O(n)}_{\text{Dodavanje čvorova}} + \underbrace{O(n)}_{\text{Dodavanje bridova}} + \underbrace{O(n)}_{\text{Pretraga stabla (BFS)}}$$

$$T_{\text{main}} = O(1) + O(n) + O(n) + O(n)$$

$$T_{\text{main}} = O(n)$$

5.6. Složenost Tree

Iako sljedeći kod definira strukturu podataka koja inicijaliira strukturu podataka, te nije dio algoritma, njena složenost će ipak biti analizirana u nastavku seminara:

```
Tree() {
    for (int i = 1; i <= MAXN; i++) { //a1, <= *(n)
        cvorovi[i].zauzet = false;  //b1
        for (int j = 1; j <= MAXN; j++) { //b2 <= *(n)
            veze[i][j] = false;  //c1
        }
    }
}
```

Konstruktor `Tree` inicijalizira podatke o čvorovima i matricu susjedstva. Prvo postavlja sve čvorove kao neiskorištene, a zatim inicijalizira matricu veza tako da nema povezanih bridova.

Vanjska petlja prolazi kroz svih n čvorova i postavlja ih kao neiskorištene, što ima složenost $O(n)$. Nakon toga, unutarnja petlja prolazi kroz svih n mogućih susjeda svakog čvora i postavlja njihove vrijednosti na false, što rezultira složenošću $O(n^2)$.

$$T_{\text{Tree}} = a_1 \cdot n + b_1 \cdot n + a_2 \cdot (n \cdot (b_2 + c_1))$$

$$T_{\text{Tree}} = d_1 \cdot n + d_2 \cdot n^2$$

$$T_{\text{Tree}} = \underbrace{O(n)}_{\text{Inicijalizacija čvorova}} + \underbrace{O(n^2)}_{\text{Inicijalizacija matrice susjedstva}}$$

$$T_{\text{Tree}} = O(n) + O(n^2)$$

Pošto je najveći član $O(n^2)$, možemo zaključiti da je složenost fukcije:

$$T_{\text{Tree}} = O(n^2)$$

5.7. Ukupna vremenska složenost

Iako struktura podataka nije direktno vezana uz algoritam složenost sam strukture će ipak biti spomenuta. Struktura podataka koristi matricu susjedstva za pohranu veza između čvorova. Inicijalizacija ove matrice prolazi kroz svih n čvorova i njihove moguće susjede, što

daje složenost $O(n^2)$. Funkcije za dodavanje čvorova i bridova imaju složenost $O(1)$.

Funkcija `main` omogućava unos podataka, dodavanje čvorova i bridova te pokreće algoritam. Dodavanje podataka i pretraga najdužeg puta zajedno imaju složenost $O(n)$.

Algoritam za pronalaženje najdužeg puta u stablu oslanja se na dvije BFS pretrage. Svaka BFS pretraga prolazi kroz sve čvorove i njihove susjede, ali budući da je stablo rijedak graf s $n-1$ bridova, složenost BFS-a ostaje $O(n)$. Budući da se u funkciji `FindLongestPath` BFS poziva dva puta, ukupna složenost algoritma ostaje $O(n)$

Dakle, konačna vremenska složenost algoritma je $O(n)$.

Tablica 4: Prikaz složenosti svih funkcija

Naziv	**Složenost**
Tree	$O(n^2)$
AddNode	$O(1)$
AddEdge	$O(1)$
BFS	$O(n)$
FindLongestPath	$O(n)$
main	$O(n)$

6. Zaključak

U ovom sminarskom radu analiziran je problem pronalaska najudaljenijih čvorova u stablu i određivanje njegovog promjera. Razvijen je algoritam temeljen na pretraživanju u širinu (BFS), koji se provodi u dva koraka. Prvo se pronalazi najudaljeniji čvor od početnog čvora, a zatim se iz tog čvora pokreće novo pretraživanje kako bi se pronašao drugi najudaljeniji čvor i izračunala udaljenost između njih.

U ovoj imlementaciji irađen je kod za traženje najduljeg puta u stablu, te je krirana struktura podataka. `Tree()` inicijalizira stablo postavljanjem svih čvorova kao nezauzetih i svih veza u matrici susjedstva na `false`, stvarajući osnovnu strukturu grafa. Funkcija `AddNode(indeks, oznaka)` omogućuje dodavanje čvora s određenim indeksom i oznakom u stablo, dok funkcija `AddEdge(u, v)` dodaje brid između čvorova u i v, te ažurira matricu susjedstva kako bi označila povezanost.

Funkcija algoritma je `BFS(pocetak, najdaljiCvor, maksUdaljenost)`, koja provodi pretragu u širinu počevši od zadanog čvora i identificira najudaljeniji čvor te njegovu udaljenost. Na temelju ove funkcije, `FindLongestPath()` koristi dvostruku BFS pretragu za pronalazak promjera stabla. Funkcija prvo pronalazi najudaljeniji čvor od proizvoljnog početnog čvora, a zatim ponovno pretražuje stablo kako bi izračunala najveću udaljenost između čvorova, tj. promjer stabla. Na kraju, funkcija ispisuje rezultate, uključujući promjer stabla i par čvorova između kojih je taj promjer ostvaren.

Teorijski dio rada obuhvatio je osnovne pojmove o grafovima, gdje se posebna pažnja posvetila stablima kao posebnom tipu grafa. Opisana su njihova svojstva, struktura itd, obrađene su metode ophodnje stabala, te njihova primjena u različitim područjima računalnih znanosti.

Praktični dio rada prikazuje implementaciju algoritma i njegovu primjenu na stvarnom primjeru, uz skicu grafa koja dodatno objašnjavaju način rada. Analizirane su glavni aspekti algorima, poput dodavanja čvorova, dodavanje bridova i pretraživanja, ali se najviše pažnje posvetilo funkciji BFS.

Složeost samog algoritma može se prikazati kroz funkciju `BFS` koja je osnovni algoritma i koristi se za pretragu stabla, prolazi kroz sve čvorove i njihove susjede. Budući da je stablo rijedak graf s $n - 1$ bridova, složenost `BFS` pretrage ostaje $O(n)$. Algoritam za pronalaženje najdužeg puta u stablu koristi dvije pretrage, pa ukupna složenost funkcije `FindLongestPath` je $O(n)$. Stoga možemo zaključiti da je konačna vremenska složenost algoritma $O(n)$.

Popis literature

[1] D. Kusalić, „Napredno programiranje i algoritmi u Cu i C++-u,” *Zagreb, Element*, 2010.

[2] J. G. Milena Vujošević Jančić, *Dinamičke strukture podataka — Stabla*, `https://matfp2.github.io/p2m/predavanja/slajdovi/11_12_stabla.pdf`, Pristupljeno: 20.01.2025.

[3] prof. dr. sc. Alen Lovrenčcić, *Binarno stablo ATP*, Prezentacija sa predavanja, 2021/22.

[4] F. M. Predrag Janičić, *Osnove programiranja kroz programski jezik C*, 2022.

[5] R. Sedgewick i K. Wayne, *Algorithms*. Addison-wesley professional, 2011.

[6] *Složenost algoritama*, `https://www.fer.unizg.hr/_download/repository/diskont1-08.pdf`, Fakultet elektrotehnike i računarstva.

Popis slika

1. Primjer grafa 2

2. Primjer stabla 6

3. Rezultat pokretanja programa 12

4. Prikaz grafa iz primjera 13

Popis tablica

1. Matrica susjedstva grafa iz grafa sa slike 1. 4
2. Popis susjedstva grafa 4
3. Prikaz funkcija algoritma 10
4. Prikaz složenosti svih funkcija 20